

```

#include <stdio.h>

#include <mpi.h>

int main(int argc, char** argv)
{
    int rank, size;

    int value;

    int sum, prod;

    int max, min;

    // Initialize the MPI environment

    MPI_Init(&argc, &argv);

    // Get rank and size

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Each rank gets a distinct initial value: rank 0 -> 1, rank 1 -> 2, rank 2 -> 3, etc.

    value = rank + 1;

    // 1. MPI_Reduce with MPI_SUM and MPI_PROD (Results available ONLY at root rank 0)

    MPI_Reduce(&value, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    MPI_Reduce(&value, &prod, 1, MPI_INT, MPI_PROD, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("\n--- Results on Root (Process 0) using MPI_Reduce ---\n");

        printf("Sum (MPI_SUM) : %d\n", sum);

        printf("Prod (MPI_PROD) : %d\n", prod);

        printf("-----\n\n");
    }
}

```

```
// 2. MPI_Allreduce with MPI_MAX and MPI_MIN (Results distributed to ALL ranks)

MPI_Allreduce(&value, &max, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
MPI_Allreduce(&value, &min, 1, MPI_INT, MPI_MIN, MPI_COMM_WORLD);


// Every rank prints its received Allreduce result

printf("Process %d -> Max (MPI_MAX): %d, Min (MPI_MIN): %d\n", rank, max, min);


// Finalize the MPI environment

MPI_Finalize();

return 0;

}
```